Pack 07 · Profiling and the GPU

How to find where the time goes before changing anything: the measure-floor-locate loop, memory snapshots, Nsight Systems and Nsight Compute, and why fusion is a memory fix. Lessons 55–58, 60, 61. Answer each question out loud in about a minute, then check the card.

Q1 · "Just turn on torch.compile" PRIORITY

Lessons: 55, 61

> **INTERVIEWER**
>
> Users say decode on our Llama 3 8B endpoint (one H100, BF16) feels slow. The manager says "just turn on torch.compile." Before changing anything, what do you measure, what floor do you compare against, and how do you decide whether compute, memory or overhead owns the missing time?

60-second answer

- **Measure one clean number:** time per output token, with a GPU synchronize before stopping the timer, warm-up runs discarded, median of several.
- **Compare with a floor:** weight bytes ÷ bandwidth = 16 GB ÷ 3.35 TB/s ≈ **4.8 ms**. **MBU** = floor ÷ measured. At 25 ms that's ≈ **19%**: most of the time is missing.
- **Locate it with a trace:** sparse kernels with gaps between them means **overhead** (CPU launches). Kernels packed back to back at 70–80% MBU means **memory**. Packed but still low MBU means **one slow kernel**: take it to Nsight Compute.
- torch.compile with `mode="reduce-overhead"` (CUDA graphs plus fusion) fixes **overhead** only. In the lesson's 7B Llama it went from 25.5 to 107 tokens/s (MBU 17% → 72%). If decode is already memory-bound, only **fewer bytes** helps (int8 weights got 157 tokens/s by raising the floor).
- **Change one thing, re-measure, log it.** The bottleneck moves after every fix.

Key numbers: floor = weight bytes ÷ bandwidth; MBU = floor ÷ measured time.

Likely follow-up: "MBU is 75% after compile. Next?" It's now memory-bound: quantize weights, or batch more users onto each weight read.

> **TRAP**
>
> I stopped at "decode should be memory-bound" without computing MBU, and couldn't say which trace pattern means overhead versus memory, or which one torch.compile actually fixes.

Q2 · OOM at hour 6 with 60 GB allocated PRIORITY

Lessons: 56

> **INTERVIEWER**
>
> Our server runs fine for hours, then dies with CUDA out-of-memory around hour 6. At the crash, `torch.cuda.memory_allocated()` says 60 GB on an 80 GB card. What could the memory snapshot show, and how can it be out of memory with 20 GB free?

60-second answer

- Record with `torch.cuda.memory._record_memory_history()`, dump with `_dump_snapshot()`, open in memory_viz. Every allocation carries its size and stack trace.
- **Healthy is a sawtooth:** a flat floor of weights and workspace; each request climbs as its KV grows (128 KB per token, ≈ 262 MB for 2,000 tokens) and falls when it finishes.
- **A leak is a climbing floor:** every valley lands higher. The stack trace on the band that never frees names the line holding the reference.
- **Fragmentation is a reserved-versus-allocated gap:** the caching allocator reserves big segments and carves tensors out of them. Varying-size, varying-lifetime tensors chop them up, so 20 GB is free but scattered and one large contiguous request fails.
- Fixes: `expandable_segments:True`, fixed-size pools (why vLLM pages its KV cache), fewer varying-size temporaries.

Key numbers: allocated = live tensors; reserved = held by the allocator; KV per token for Llama 3 8B = 128 KB.

Likely follow-up: "Why didn't a too-long request cause it?" That fails on the first such request, not after 6 hours, and a real engine preallocates its KV cache, so it can't grow.

> **TRAP**
>
> I guessed "the KV cache exploded" or "a request was over max length". Neither fits a crash after hours, and neither explains OOM with 20 GB free.

Q3 · A clean Nsight Systems trace of a live server PRIORITY

Lessons: 57

> **INTERVIEWER**
>
> You run `nsys profile` on a live vLLM server. The file is huge and mostly model loading, and during decode the GPU row shows one opaque block per step. How do you fix each problem, and what three things do you read off a clean decode window?

60-second answer

- **Window:** `--delay=180 --duration=120` skips loading and cold requests and records steady traffic; `--kill=none` keeps the server alive after collection.

- **Opaque block:** each step is a replayed **CUDA graph**. `--cuda-graph-trace=node` opens it into kernels (with noticeable overhead). Also set `VLLM_WORKER_MULTIPROC_METHOD=spawn`, because nsys can't follow a forked worker.
- **Read 1, gaps:** holes in the GPU stream while users wait mean the device is idle, not a kernel problem. A quiet API lane above means nothing was asked; a busy one means work didn't arrive in time.
- **Read 2, pacing:** a packed launch lane over thin slivers means the CPU sets the pace; shoulder-to-shoulder kernels wider than their launch calls mean the GPU does.
- **Read 3, copies:** steady decode keeps the memory rows nearly silent. Wide or rhythmic host↔device copies mean state crosses PCIe every step.

Key numbers: `--trace=cuda,nvtx,osrt`; NVTX ranges ("prefill", "decode") label phases on the timeline.

Likely follow-up: "When do you switch to Nsight Compute?" Only once the GPU is busy edge to edge and one named kernel owns the time.

> **TRAP**
> I hadn't used Nsight Systems and couldn't name the window or CUDA graph flags, or the three readings.

Q4 · "Occupancy is terrible, retune the block size" PRIORITY

Lessons: 58, 61

> **INTERVIEWER**
> Nsight Compute on our decode GEMM shows a duration 3× longer than in production, memory throughput 92%, compute 2%, achieved occupancy 25%. A teammate wants to retune the block size. What do you tell them?

60-second answer

- **The duration is an artifact.** `ncu` replays the launch once per metric group (46 passes by default), restores memory between passes, serializes launches and pins clocks. Only the metric values count.
- **92% memory, 2% compute is the healthy answer.** A decode GEMM has intensity ≈ 1 FLOP per byte, far below the ridge of ≈ 295, so it should sit at the memory limit with compute idle. It's at its floor.
- **Low occupancy doesn't convict a kernel.** Extra warps only hide latency; with memory already near its ceiling they'd have nothing to fetch. Worry when occupancy is low **and** both throughputs are low.
- Retuning buys nothing. Only fewer bytes (quantized weights) or more tokens per weight read (batching) speed it up.
- Predict the Speed Of Light numbers from the roofline **before** opening the report, so a disagreement means something.

Key numbers: ridge = 989 TFLOPS ÷ 3.35 TB/s ≈ 295; decode GEMM I ≈ batch size; decode attention I ≈ 4 with GQA.

Likely follow-up: "What if the L2 hit rate on that GEMM is high?" Some bytes you counted never left the cache, so redo the bound before trusting the report (16 GB of weights can't loop in a 50 MB L2).

> **TRAP**
>
> I hadn't used Nsight Compute and couldn't read the report: replay time, the Speed Of Light pair and occupancy all needed explaining.

Q5 · Three elementwise ops in eager mode PRIORITY

Lessons: 60, 61

> **INTERVIEWER**
>
> In eager PyTorch a layer runs `x = x * scale`, `x = x + bias`, `x = gelu(x)` on a big BF16 activation. Why is it slow when the math is trivial? What does fusing it into one kernel change, which bucket does that target, and where do the intermediates live?

60-second answer

- Every eager op **starts and ends in HBM**: read x, write x. Three ops are **three round trips** (6 tensor-sized transfers) for a few FLOPs per element.
- Intensity is ≈ 0.25 FLOP per byte for each simple op (1 FLOP per 2 bytes read + 2 written), far below the ridge of ≈ 295, so each kernel is **memory-bound**.
- Fusion does one read and one write: ≈ **3× less traffic**, so ≈ 3× faster. It targets the **memory** bucket (and overhead, 3 launches → 1).
- It is **still memory-bound** afterwards: even with GELU's extra math the fused intensity is a few FLOP per byte, nowhere near 295.
- Intermediates stay in each thread's **registers** (≈ 1 cycle, 256 KB per SM); tiled ops like matmul and FlashAttention stage data in **shared memory**.

Key numbers: 4096 × 4096 BF16 = 33.5 MB; one read + write = 67 MB ÷ 3.35 TB/s ≈ 20 μs against 0.25 μs of math (≈ 80×).

Likely follow-up: "When would fusion make a kernel compute-bound?" Only when it fuses into something that already reuses each byte hundreds of times, like a matmul epilogue; elementwise chains alone never reach the ridge.

> **TRAP**
>
> I said fusion moves the op to compute-bound. It cuts bytes, but the intensity stays tiny, so the kernel is still memory-bound, just faster.

Build it yourself

No exercise for this pack yet. The best practice is real: time a decode step with and without `torch.compile(mode="reduce-overhead")`, compute MBU for each, and capture one Nsight Systems window of each to see the gaps disappear.